KARACHI AQI PREDICTOR

End-to-End Serverless Machine Learning Forecasting System

Final Project Report

Ruhab Iqbal
10 Pearls

Submission date: 2 September 2026

Live application
https://karachi-aqi-forecast-sypfn2an46wrsyxksldyz3.streamlit.app/

Repository
https://github.com/k230743-eng/karachi-aqi-forecast

Executive Summary

This project implements the Pearls AQI Predictor brief as a working 72-hour forecasting service for Karachi. The final result is not only a trained model: it is a complete data and ML lifecycle that collects live inputs, repairs missed ingestion windows, stores engineered features in Hopsworks, generates delayed training labels, retrains 72 horizon-specific XGBoost models every day, registers new model versions, produces an hourly three-day forecast, and publishes that forecast through a deployed Streamlit application.

The historical backfill uses Open-Meteo air-quality and weather data from August 2022 through June 2026. After cleaning and feature engineering, 34,051 complete hourly rows remain. Exploratory analysis showed strong temporal structure, a right-skewed AQI distribution, a pronounced winter seasonal effect, an evening AQI peak, a strong PM2.5 relationship, and negative associations with wind. These findings directly influenced the final features: long AQI and particulate lags, rolling summaries, change and volatility features, time indicators, and target-hour weather.

Several forecasting approaches were evaluated rather than selecting a production algorithm in advance. Persistence and mean baselines established difficulty by horizon; Ridge Regression and Random Forest showed that useful signal existed well beyond the immediate future; a TensorFlow dense network was also tested; and gradient-boosting experiments led to a direct XGBoost design with one model per forecast hour. In the latest Hopsworks metrics supplied with the project, the 72-hour XGBoost model records MAE 12.57, RMSE 17.03 and R^2 0.380, while the one-hour model records MAE 0.86, RMSE 1.72 and R^2 0.994.

The system is automated with GitHub Actions. The live pipeline runs hourly; the training lifecycle runs daily. Hopsworks acts as the persistent Feature Store and Model Registry, while Streamlit Community Cloud serves the dashboard. This architecture satisfies the brief while remaining reproducible and operational without a continuously running local machine.

Report Map

1. Project Brief and Scope

The original Pearls AQI Predictor brief asks for a three-day AQI prediction service built on a serverless stack. It specifies an external weather/pollution data source, feature and target generation, a Feature Store, historical backfill, model experimentation across Scikit-learn and deep-learning approaches, MAE/RMSE/R^2 evaluation, a Model Registry, hourly feature automation, daily training automation, an interactive web application, EDA, explainability with SHAP or LIME, and hazardous-AQI alerts.

The project was scoped to Karachi and implemented as a 72-hour hourly forecast. A deliberate choice was made to preserve the full project lifecycle in individual scripts rather than collapsing everything into one notebook. This makes the development history visible and separates historical experimentation from production ingestion, training, inference and presentation.

2. System Architecture

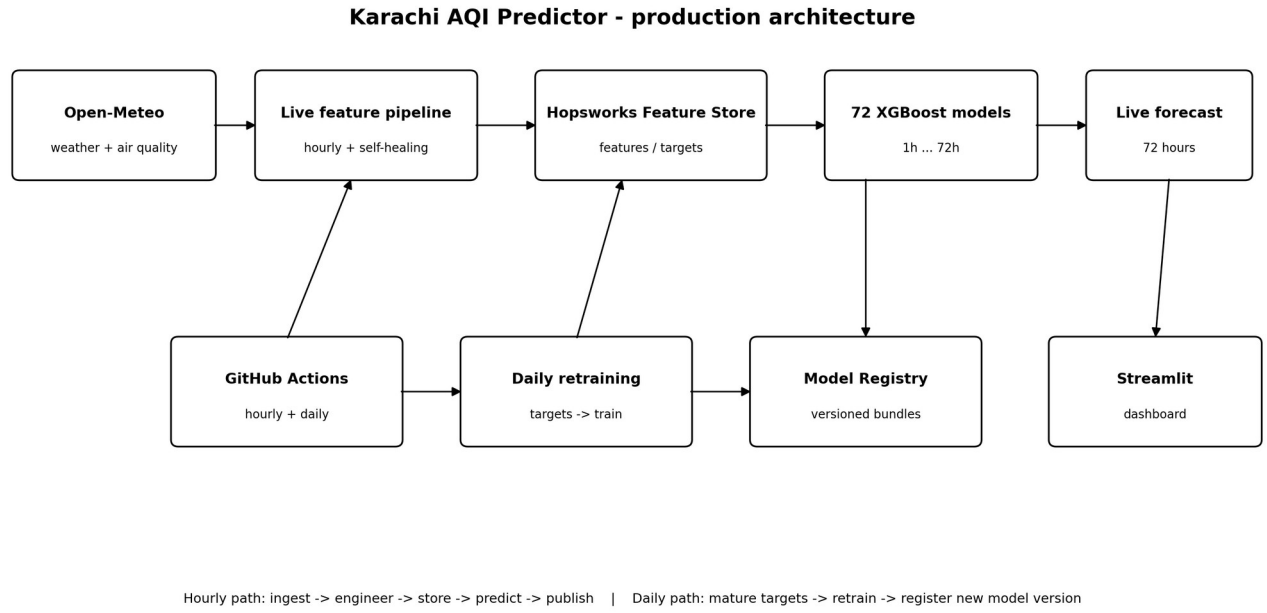


Figure 1. Final production architecture and the two automation paths.

There are two connected loops. The hourly loop keeps features and forecasts current. The daily loop waits for labels to mature, retrains the models, and registers a new version. The live predictor then selects the newest registry version automatically, which closes the retraining-to-serving loop.

The deployed dashboard intentionally does not import the Hopsworks client. During deployment work, the current Hopsworks and Streamlit dependency stacks required incompatible protobuf ranges. The stable solution was to keep the ML pipeline and dashboard environments separate. The hourly workflow publishes the latest forecast CSV to GitHub, and Streamlit Community Cloud uses the repository as its source. This preserves a lightweight web deployment while the Feature Store and Model Registry remain authoritative for the ML pipeline.

3. Data Collection and Historical Backfill

The data collector uses Karachi coordinates 24.8607, 67.0011 with the Asia/Karachi timezone. Open-Meteo provides two hourly feeds: air-quality variables and archived weather variables. The initial backfill spans 1 August 2022 to 30 June 2026 and produces 34,320 rows from each source before cleaning.

Source	Variables retained	Historical rows
Air quality	PM10, PM2.5, CO, NO2, SO2, ozone, dust, aerosol optical depth, US AQI	34,320
Weather	temperature, humidity, dew point, pressure, precipitation, cloud cover, wind speed/direction/gusts	34,320

The two hourly tables are merged on timestamp. This design lets the same weather and pollution schema drive both historical training and the live feature pipeline. The project brief named AQICN and OpenWeather as examples rather than fixed requirements, so Open-Meteo was selected after API exploration because it could provide the required hourly variables and historical coverage.

A practical limitation is that the selected Open-Meteo air-quality feed is model-based rather than a dedicated Karachi regulatory ground-station dataset. The results should therefore be read as forecasts of the selected US AQI data product, not as certified station observations. This limitation is carried through to the final discussion rather than hidden behind the model metrics.

4. Data Quality and Preparation

The preparation pipeline first removes rows missing essential pollutant fields, sorts and deduplicates by timestamp, and explicitly checks for gaps larger than one hour. This check is important because the feature engineering uses row-based shifts: shift(24) is only a 24-hour lag when every hourly timestamp is present.

After the initial merge and cleaning, 34,219 rows with 19 raw variables remain. The 168-hour maximum lag then causes the first week of engineered rows to be discarded, leaving 34,051 complete hourly rows and 72 stored feature columns including time. The final historical feature range is 12 August 2022 05:00 through 30 June 2026 23:00; the live pipeline extends the Hopsworks group beyond this static backfill.

Extreme AQI values were not automatically removed. The highest historical AQI in the analysis is 297, and severe episodes are uncommon but meaningful. Removing them solely because they are rare would make the training set less representative of the events the application is expected to warn about.

5. Exploratory Data Analysis

EDA was used as a decision tool, not only as a visualization requirement. The main question was which structures would matter at horizons ranging from one hour to three days.

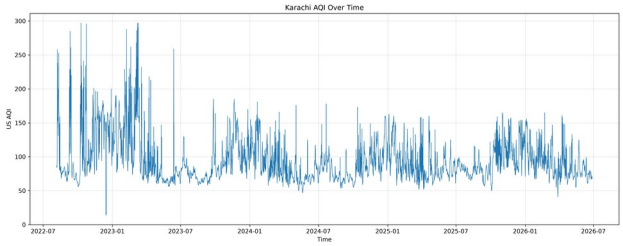


Figure 2a. AQI over the historical period.

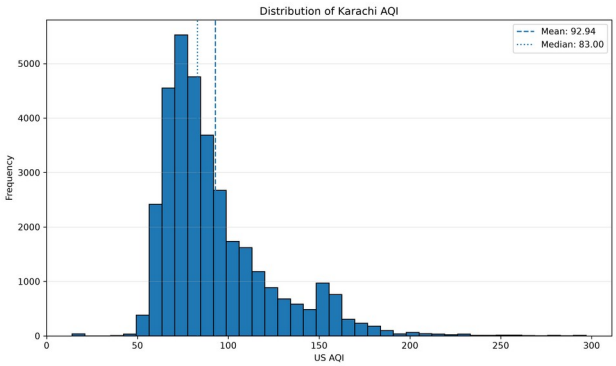


Figure 2b. AQI distribution.

The time series contains repeated short-lived pollution spikes and visible seasonal changes. AQI is right-skewed: the engineered historical data has mean 92.63, median 83, and maximum 297. Most values lie roughly between 60 and 110, so extreme episodes are a minority. This makes MAE useful for typical error, while RMSE remains important because it penalizes larger misses during spikes.

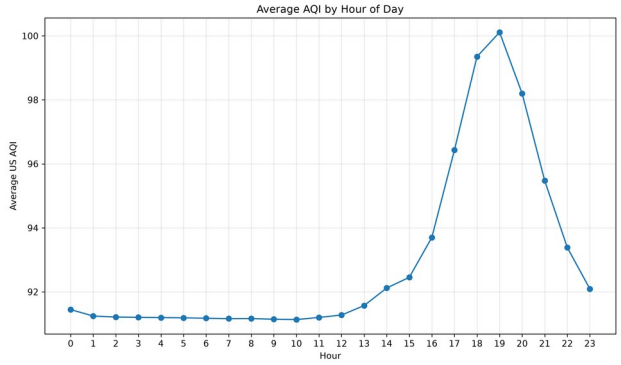


Figure 3a. Average AQI by hour.

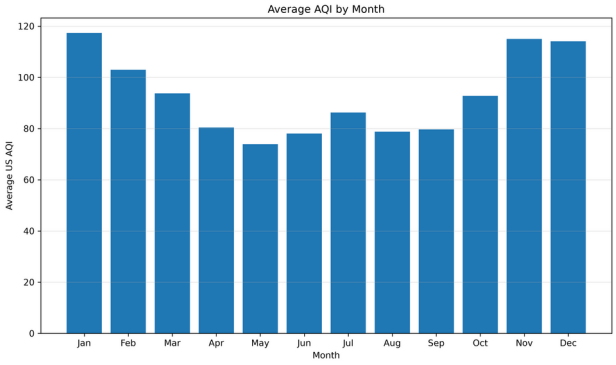
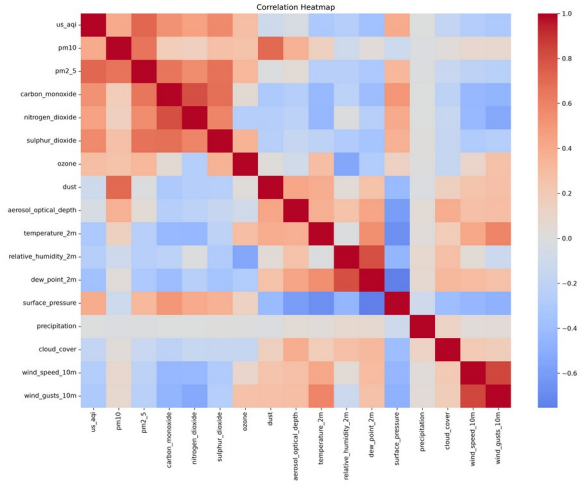


Figure 3b. Average AQI by month.

The daily profile stays relatively flat overnight and through the morning, then rises strongly in the late afternoon and peaks around 19:00. The monthly profile is much stronger than the weekday effect: January is the most polluted average month in the sample, November and December are also high, and May is the lowest. These patterns justified keeping hour and month as explicit target-time features.



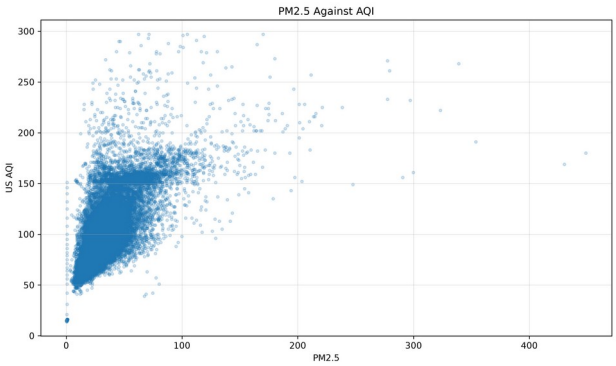


Figure 4a. Correlation structure across pollutants and weather.

Figure 4b. PM2.5 against US AQI.

PM2.5 has the strongest positive linear relationship with AQI in the exploratory analysis, approximately 0.725, but the scatter is visibly nonlinear and wide. PM10 is also useful but weaker. Wind speed is negatively associated with AQI and severe AQI observations are concentrated at low-to-moderate wind speeds. Humidity is weaker and highly scattered. Precipitation has almost no overall linear correlation.

One important modelling decision came from this: low-correlation weather variables were not removed simply from the heatmap. A tree model can exploit interactions and thresholds that a Pearson correlation coefficient cannot show, and the later SHAP analysis confirmed that weather becomes especially important at longer horizons.

6. Feature Engineering Decisions

The final feature design combines current state, memory, trend, volatility and target-hour context. The historical table stores 71 model inputs plus the timestamp; every horizon then adds 14 target-hour inputs, producing 85 features presented to each XGBoost model.

Feature family	Definition	Count
Current pollution	8 pollutant variables + current US AQI	9
Current weather	temperature, humidity, dew point, pressure, precipitation, cloud, wind speed/direction/gusts	9
Calendar	hour, day of week, month, weekend	4
AQI lags	1, 3, 6, 12, 24, 48, 72, 168 h	8
PM2.5 lags	1, 3, 6, 12, 24, 48, 72 h	7
PM10 lags	1, 3, 6, 12, 24, 48, 72 h	7
Rolling means	AQI, PM2.5, PM10 over 3/6/12/24/48/72 h	18
AQI changes	1, 3, 6, 12, 24, 48, 72 h	7
AQI volatility	24 h and 72 h rolling standard deviation	2
Target-hour inputs	8 future weather + 2 wind direction encodings + 4 calendar fields	14

The 168-hour AQI lag gives each prediction access to the previous week. Rolling means summarize persistence without requiring the model to reconstruct it from many individual hourly lags. Change features capture direction, while rolling standard deviation distinguishes stable periods from volatile ones. Future wind direction is encoded with sine and cosine because 359 degrees and 1 degree should be close numerically even though their raw degree values are far apart.

EDA suggested cyclical encodings for hour and month as candidates. A dedicated Ridge 72-hour experiment compared original features, cyclical replacement, and original-plus-cyclical variants across regularization strengths. The best saved experiment did not improve on the stronger existing Ridge result, so a wholesale cyclical replacement was not carried into the final production feature set. This is an example of an EDA idea being tested rather than adopted automatically.

7. Model Development and Comparison

The modelling process was intentionally staged. The persistence baseline asks a hard question: can a learned model beat simply using the current AQI as the future AQI? A training-mean baseline provides a second sanity check. Ridge Regression then measures how much the engineered features help a regularized linear model, while Random Forest tests nonlinear interactions. A TensorFlow dense network satisfies the deep-learning experiment in the brief and tests whether a generic feed-forward architecture adds value. Gradient boosting was explored after these comparisons.

72-hour model	MAE	RMSE	R^2
Persistence baseline	18.82	25.20	-0.181
Ridge Regression	15.25	19.80	0.270
Random Forest	16.61	21.43	0.145
TensorFlow dense network	16.37	21.54	0.137
Direct XGBoost (initial final)	14.44	18.98	0.333
XGBoost (Hopsworks retrain)	12.57	17.03	0.380

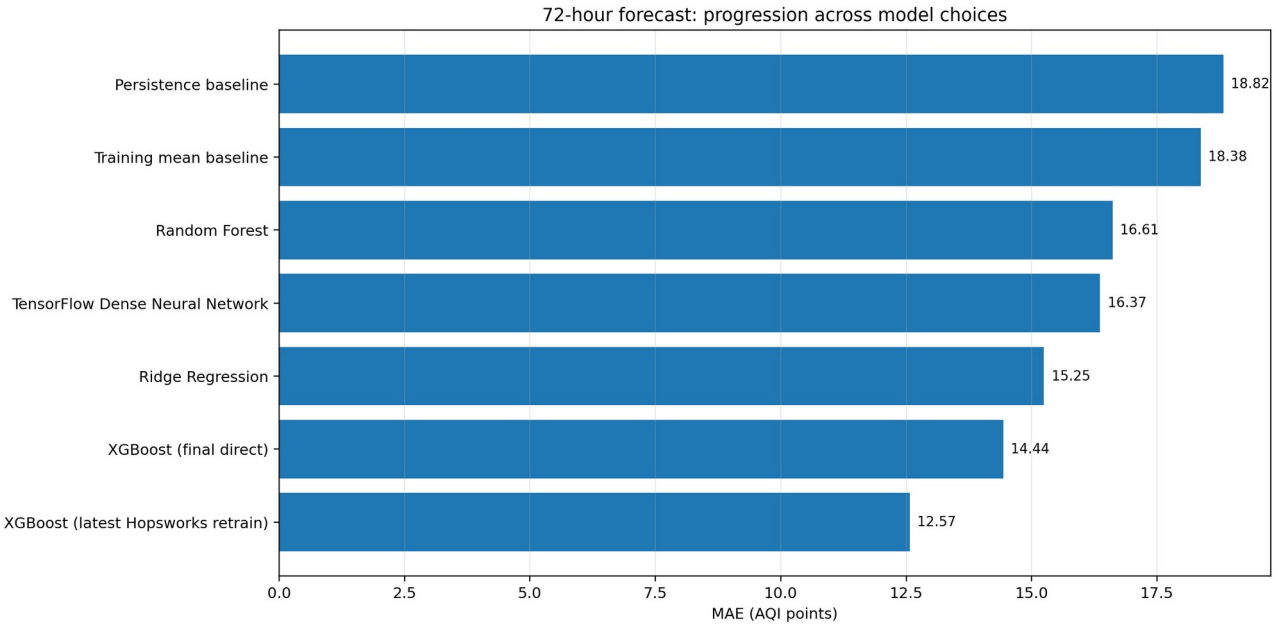


Figure 5. 72-hour MAE across the main model progression.

Persistence is excellent at one hour but deteriorates quickly: its 72-hour R^2 is -0.181, meaning it is worse than a simple mean prediction at that horizon. Ridge moves the 72-hour R^2 to 0.270. Random Forest is the strongest early model at one and six hours, but its 72-hour R^2 is only 0.145. The TensorFlow model records 0.137 at 72 hours, so complexity alone did not improve generalization.

The direct XGBoost system is the first approach that improves both the long-horizon error and the full horizon curve consistently. The initial 72-hour XGBoost model records MAE 14.44 and R^2 0.333. The later

Hopsworks retraining artifact records MAE 12.57 and R^2 0.380 at 72 hours. These results motivated the final production choice.

8. Final XGBoost Forecasting Strategy

The production design trains 72 independent XGBoost regressors, one for each hour ahead. This direct multi-horizon strategy avoids recursively feeding predictions back into the model, which would allow errors to compound through the three-day window. Every model sees the same current/historical state but receives future weather and calendar features aligned to its own target hour.

The XGBoost configuration is deliberately conservative: 800 trees, learning rate 0.02, maximum depth 2, minimum child weight 20, row subsampling 0.8, column subsampling 0.7, L1 regularization 1.0 and L2 regularization 20.0, using the histogram tree method. This is designed to capture nonlinear relationships without allowing the model to grow deep trees around rare pollution episodes.

Horizon (h)	MAE	RMSE	R^2
1	0.862	1.723	0.994
6	2.744	4.471	0.958
12	5.112	7.906	0.867
24	9.117	13.020	0.640
48	11.899	16.325	0.433
72	12.571	17.034	0.380

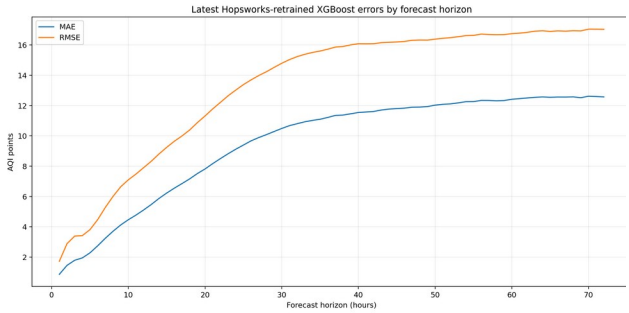


Figure 6a. Latest MAE/RMSE by horizon.

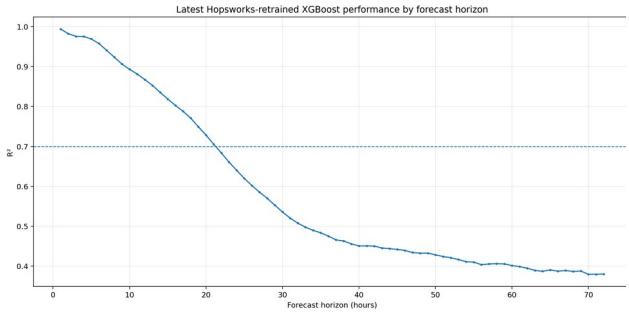


Figure 6b. Latest R^2 by horizon.

Across all 72 horizons in the supplied Hopsworks metrics file, mean MAE is 9.54, mean RMSE is 13.35, and mean R^2 is 0.580. R^2 remains at or above 0.70 through hour 21. The decline with horizon is expected and, importantly, is smooth rather than erratic.

Offline training uses historical weather shifted to the target hour, while live inference uses an actual weather forecast for the future target hour. This is a practical train/serve mismatch: realized weather is cleaner than a forecast of weather, so offline metrics are likely somewhat optimistic. It is documented as a limitation and a clear future improvement is to train on archived historical weather forecasts.

9. Explainability with SHAP

SHAP explanations were generated at six representative horizons: 1, 6, 12, 24, 48 and 72 hours. For each horizon the project saves a global mean-absolute SHAP bar plot, a beeswarm plot showing direction as well as magnitude, and a local waterfall explanation.

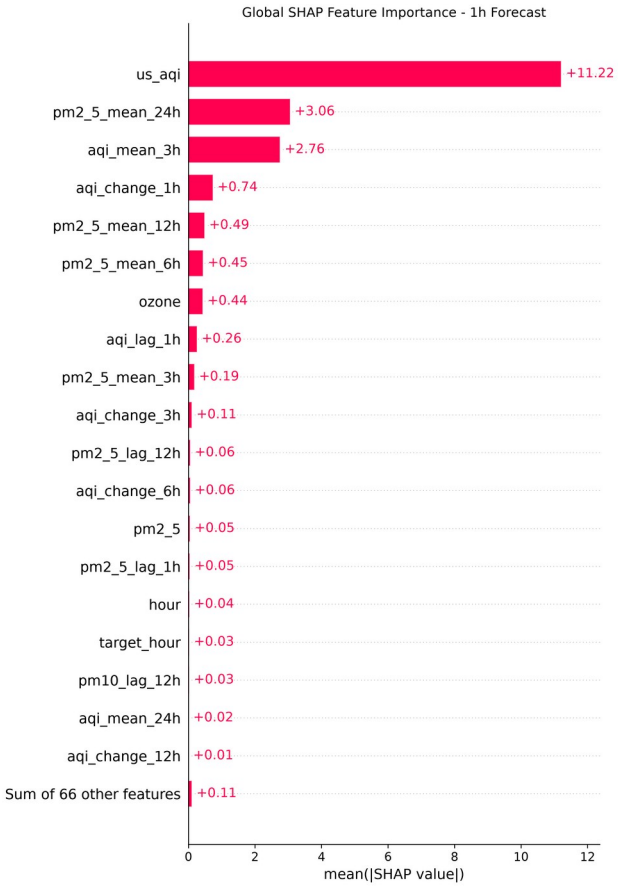


Figure 7a. Global SHAP importance at 1 hour.

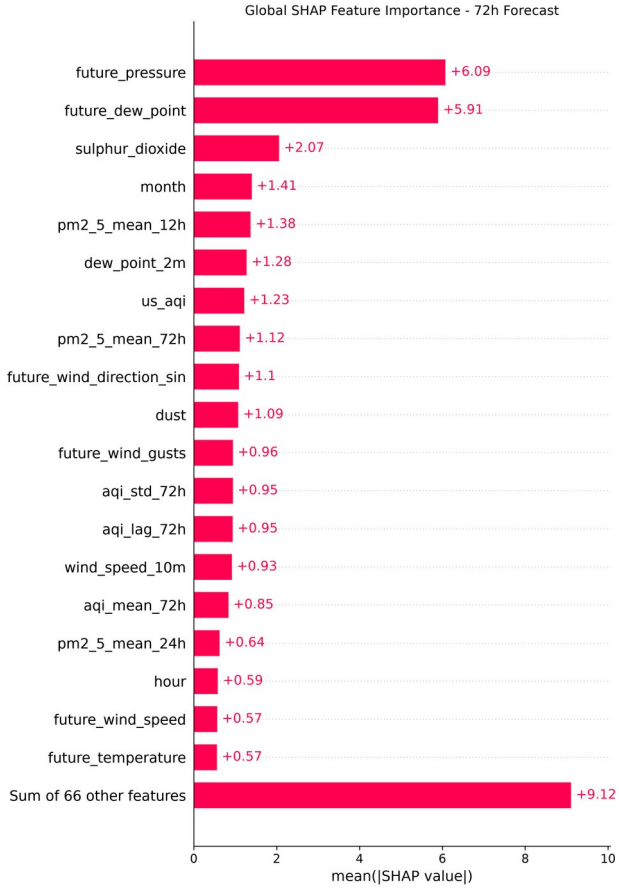


Figure 7b. Global SHAP importance at 72 hours.

The one-hour model is dominated by current AQI, with mean absolute SHAP magnitude around 11.22 in the saved explanation. The next strongest features are recent PM2.5 and short AQI summaries. This is exactly what should happen for a near-term forecast: the atmosphere has strong persistence over one hour.

At six hours, the 24-hour PM2.5 rolling mean becomes the strongest saved SHAP feature. By 24 hours, PM2.5, current AQI, recent PM2.5 averages and future dew point all contribute materially. The 72-hour model looks different again: future pressure and future dew point are the two strongest global features, followed by sulphur dioxide, month and longer PM2.5 summaries. The model therefore transitions from immediate persistence toward meteorology and seasonal context as the horizon grows.

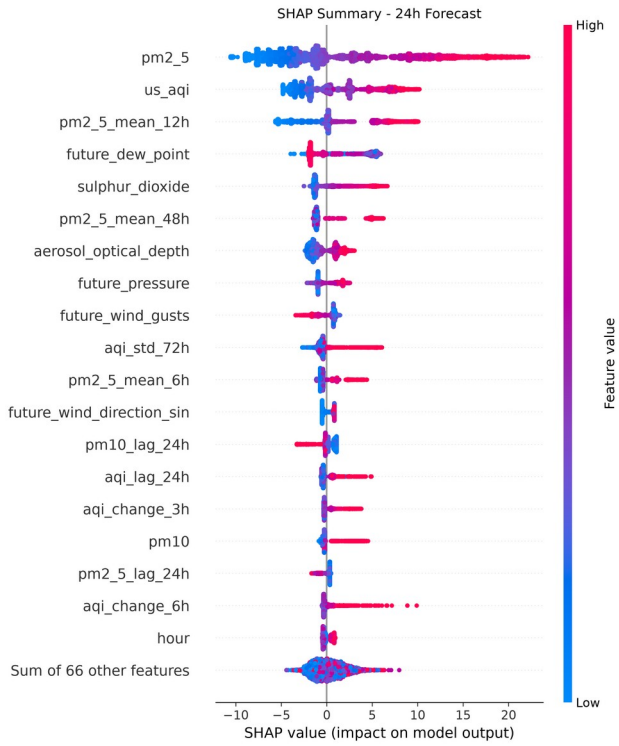


Figure 8a. SHAP direction and magnitude at 24 hours.

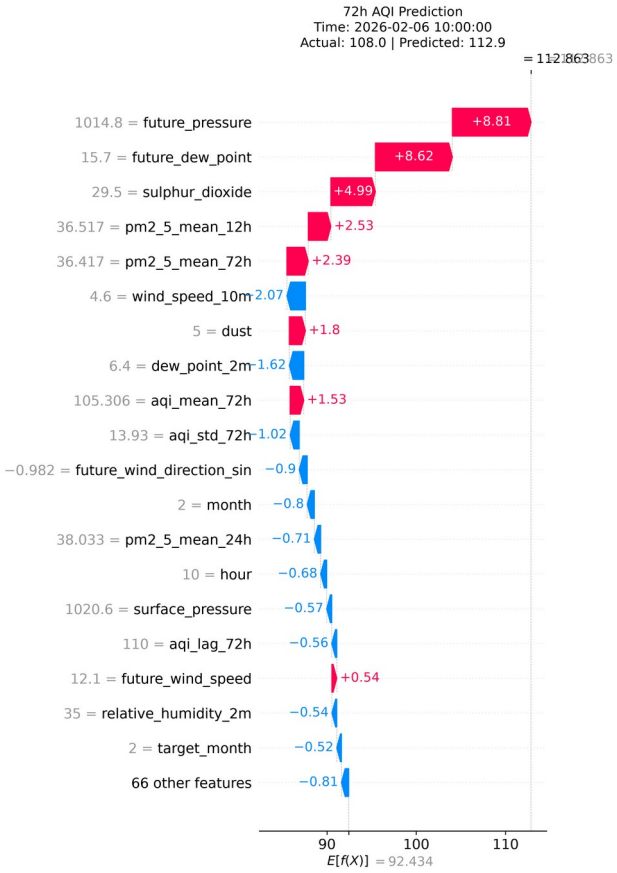


Figure 8b. Example local explanation at 72 hours.

This horizon shift is one of the strongest qualitative checks in the project. A 72-hour model that remained almost entirely dependent on current AQI would be suspicious. Instead, the explanation shows that the model changes what it relies on as the task becomes longer-term.

10. Hopsworks Feature Store and Model Registry

Hopsworks provides the persistent ML state. The current production feature group is `karachi_aqi_features` version 2, stored in Hudi format. It contains the historical and live engineered feature schema. Targets are stored separately in `karachi_aqi_targets` version 1, with time plus 72 target columns. A separate prediction feature group also stores live forecast rows.

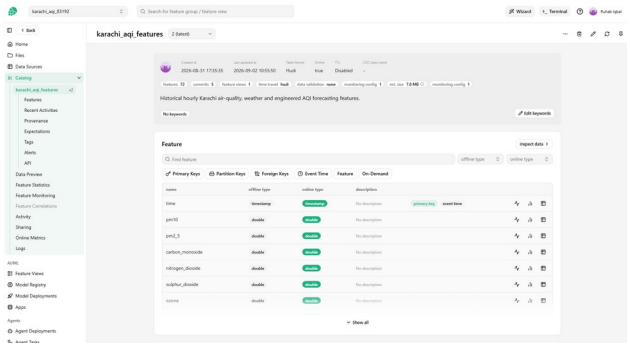


Figure 9a. Hopsworks feature group: karachi_aqi_features v2.

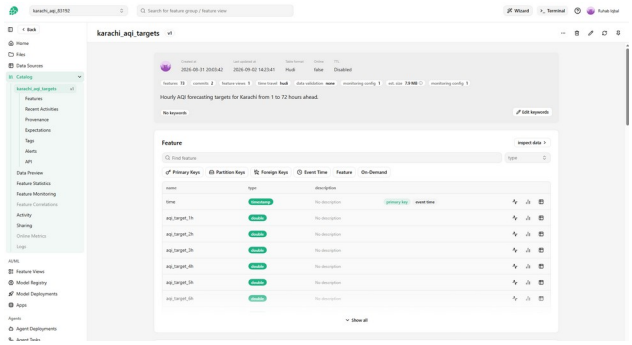


Figure 9b. Hopsworks target group: karachi_aqi_targets v1.

Separating targets from features allows delayed labels to be written only after the future AQI values are actually known. During experimentation a Feature View was also created to join features and labels. For automated retraining, the final training script reads the latest feature and target groups directly so that it does not repeatedly train on an old materialized dataset snapshot.

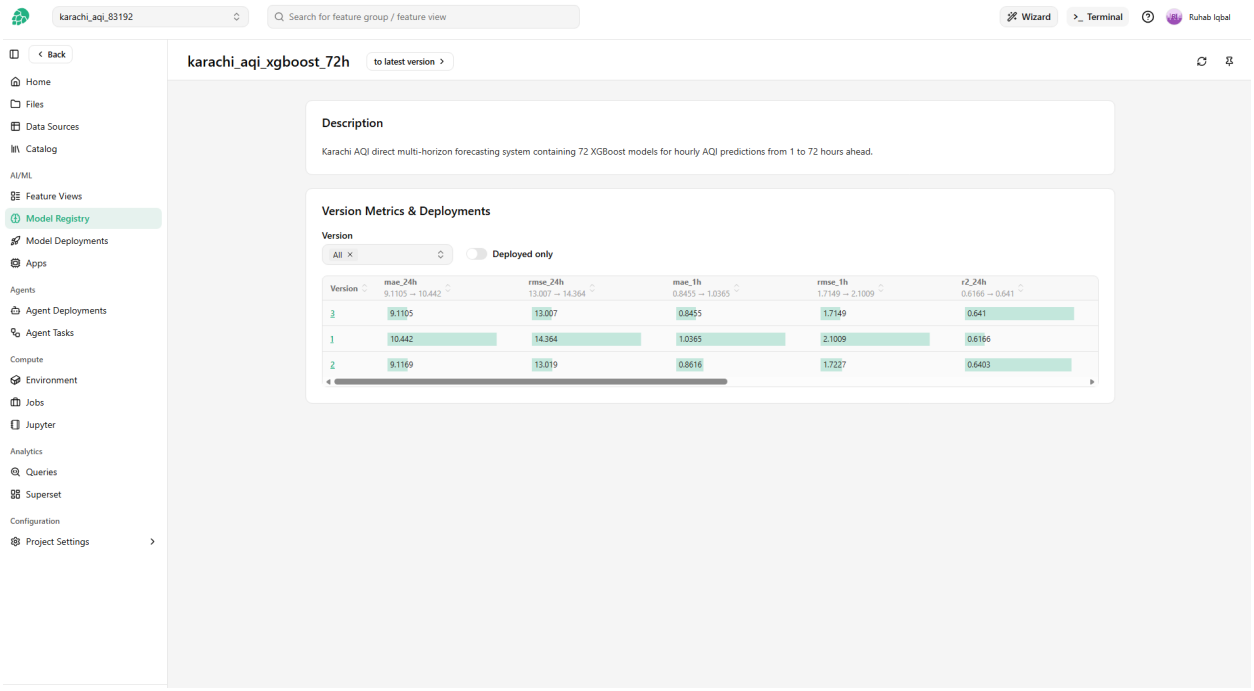


Figure 10. Hopsworks Model Registry showing multiple versions of the 72-model XGBoost bundle.

The 72 models are registered as one ZIP artifact rather than 72 independent registry objects. The bundle contains all joblib models, the exact feature-column order, metrics and metadata. This solved an earlier practical registry problem in which uploading many separate files failed partway through. Every daily retraining creates a new registry version, and the live predictor selects the maximum available version automatically.

11. Self-Healing Live Feature Pipeline

The hourly ingestion script is designed around failure recovery. It does not store a local "last successful run" flag and assume that all prior jobs completed. Instead, it queries Hopsworks for the latest stored timestamp and compares that with the latest fully completed hour in Karachi.

The script then computes two starting points: the first missing timestamp and the beginning of a 48-hour refresh window. It takes whichever is earlier, then fetches a 192-hour lookback before that point. The 192-hour context exceeds the longest 168-hour feature lag and leaves a buffer for safe reconstruction. API requests are split into 30-day chunks and use retry/backoff for HTTP 429 and common 5xx failures.

Before any row is uploaded, the script checks hourly continuity, recreates the exact feature schema, casts fields to the Hopsworks types, validates nulls and duplicate timestamps, then performs an upsert. The consequence is simple but important: if GitHub Actions is unavailable for several hours, the next successful run repairs the missing interval automatically. Recent 48-hour rows are also recalculated in case the upstream data source revises near-real-time values.

12. Delayed Targets and Daily Retraining

A production forecasting system cannot write tomorrow's AQI target today. The target updater therefore treats labels as delayed data. If the newest available AQI timestamp is t, only rows through t - 72 hours are eligible to receive a complete set of +1h through +72h targets.

For every newly mature base timestamp, `16_update_targets.py` maps the known future AQI values into `aqi_target_1h` through `aqi_target_72h` and writes those rows to the target group. Requiring all 72 labels keeps the training table simple and prevents partial-label edge cases.

The daily training job then reads the latest features and targets, joins them on time, and applies a chronological 80/20 split. The newest 20% is kept for evaluation. All 72 XGBoost models are retrained, metrics are regenerated, the artifact bundle is rebuilt, and a new Model Registry version is created. Temporary Arrow Flight read failures are retried so that a transient query-service connection drop does not immediately kill the entire daily job.

13. Live Prediction and Dashboard

Live prediction begins with the newest feature row in Hopsworks. Open-Meteo is queried for four forecast days so that every required +1h to +72h weather timestamp is available. The predictor verifies that the saved `feature_columns.json` exactly matches the hard-coded production schema before it loads any model. For each horizon it combines the same latest historical state with the weather and calendar data for that horizon's target timestamp.

Predictions are clipped at zero because AQI cannot be negative. The resulting 72 rows include generated time, forecast time, horizon, predicted AQI, forecast weather and an AQI category. They are saved both to a Hopsworks prediction feature group and to the CSV consumed by the dashboard.

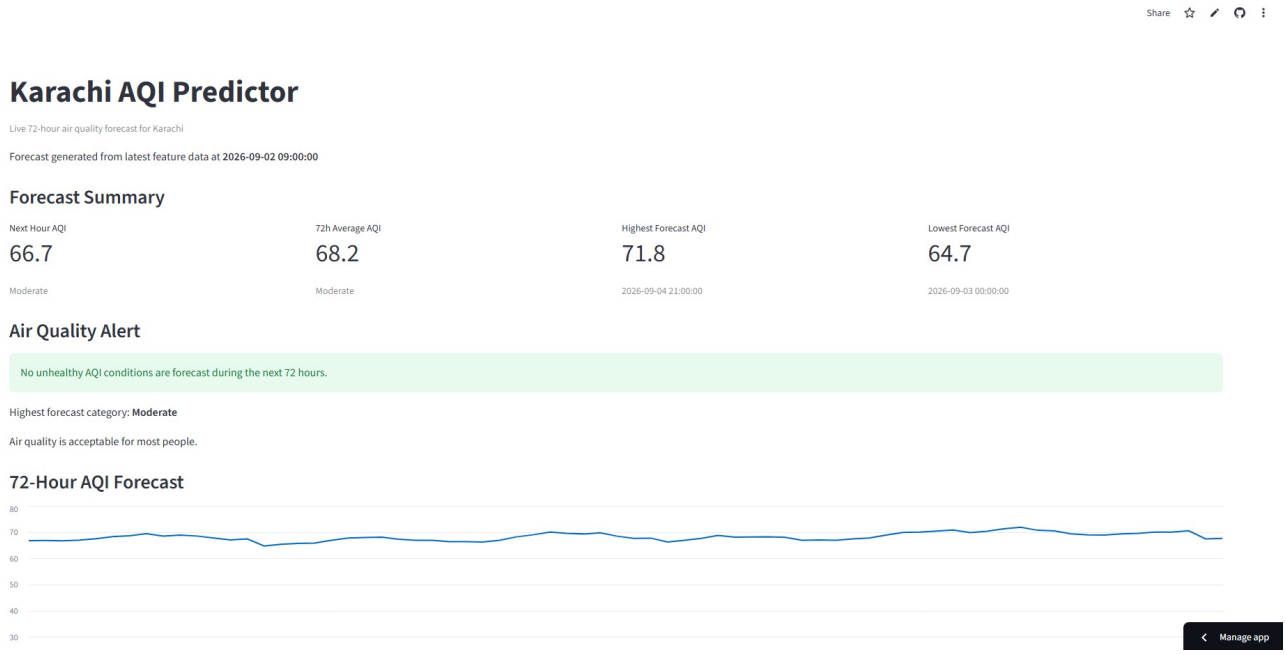


Figure 11. Deployed Streamlit dashboard using the live 72-hour forecast generated on 2 September 2026.

The supplied live forecast was generated from feature data at 2026-09-02 09:00:00. Its 72-hour average is 68.2; the minimum is 64.7 and the maximum is 71.8. All 72 rows in that snapshot fall in the Moderate

category, so the dashboard correctly shows no unhealthy-condition alert. The alert logic escalates for Unhealthy for Sensitive Groups, Unhealthy, Very Unhealthy and Hazardous forecast values.

The public application is available at: https://karachi-aqi-forecast-sypfn2an46wrsyxksldyz3.streamlit.app/

14. Automation and Deployment

GitHub Actions provides the serverless scheduler. The live workflow runs hourly. It installs only the dependencies required for live ingestion and inference, authenticates to Hopsworks with repository secrets, updates features, generates the forecast, validates that 72 output rows exist, and commits the updated forecast CSV back to the repository. The dashboard deployment then receives repository updates through Streamlit Community Cloud.

A second workflow runs daily. It updates mature targets, retrains the 72-model bundle and registers the new model version. Local Windows execution uses the Hopsworks certificate folder, while GitHub's Linux runner uses `HOPSWORKS_API_KEY`, `HOPSWORKS_HOST` and `HOPSWORKS_PROJECT`. This environment-aware login logic removed hard-coded Windows paths from the automation path without breaking local development.

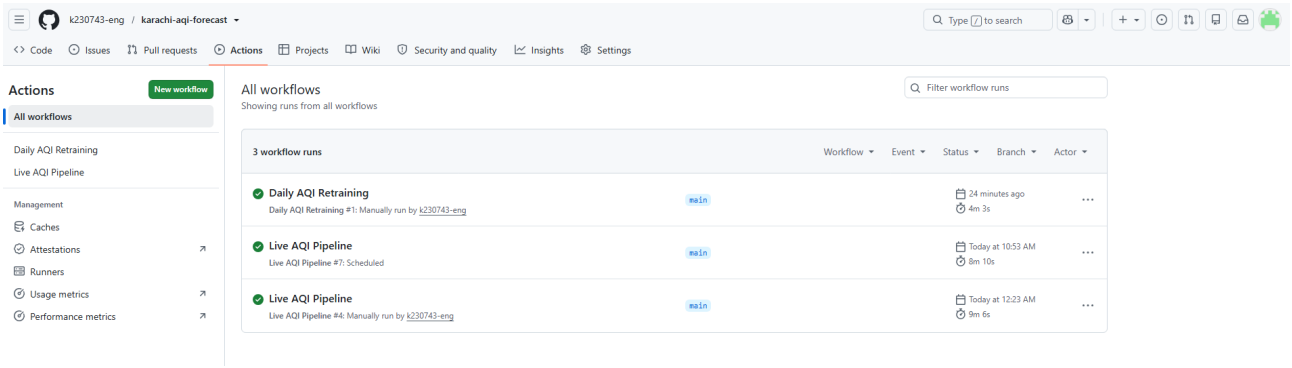


Figure 12. Successful GitHub Actions runs for both the hourly live pipeline and daily retraining workflow.

15. Engineering Challenges and Resolutions

Several of the most important project decisions came from integration problems rather than model tuning. Recording them matters because they explain why the final repository has separate environments, a ZIP registry artifact and retry logic.

Problem encountered	Resolution used in the final system
Initial Hopsworks Delta feature-group attempt did not materialize usable data.	Moved the production feature group to Hudi v2 and kept the empty v1 out of the live path.
Uploading dozens of registry files failed partway through.	Packed 72 models + configuration + metrics + metadata into one ZIP registry artifact.
Streamlit 1.63 and Hopsworks 5.0.6 required incompatible protobuf ranges.	Separated pipeline and dashboard virtual environments; deployed dashboard reads the published forecast CSV.
GitHub Actions runner lacked Kafka client required by Hopsworks inserts.	Added confluent-kafka to the live workflow requirements.
Windows certificate/temp paths do not exist on GitHub Linux runners.	Added environment-aware Hopsworks login and GitHub repository secrets.

Hopsworks Arrow Flight read occasionally ended the TCP stream.	Added longer read timeout and automatic retry/backoff to retraining reads.
Daily model registration would be useless if inference stayed on v1.	Live predictor now queries all registry versions and selects the newest.
Hourly jobs may be missed or upstream recent data may change.	Live feature pipeline finds gaps, backfills automatically, and refreshes the most recent 48 hours.

These changes are also why the final implementation differs from a minimal tutorial architecture. The project was repeatedly adjusted around actual failure modes until the complete pipeline could run unattended.

16. Requirements Coverage

Brief requirement	**Implemented result**
Predict AQI for next 3 days	72 direct hourly XGBoost forecasts.
External weather/pollution API	Open-Meteo historical/live air quality and weather.
Time and derived features	Calendar, lags, rolling means, AQI change and volatility.
Feature Store	Hopsworks feature group v2 plus target/prediction groups.
Historical backfill	Aug 2022-Jun 2026 backfill; live gap recovery thereafter.
Random Forest / Ridge experiment	Both evaluated against persistence and mean baselines.
Deep-learning experiment	TensorFlow dense neural network evaluated at 72 hours.
MAE, RMSE, R^2	Saved for baseline models and every XGBoost horizon.
Model Registry	Versioned Hopsworks model bundle containing 72 models.
Feature script every hour	GitHub Actions live workflow.
Training every day	GitHub Actions daily target/retrain/register workflow.
EDA	Time, distribution, calendar, correlation and scatter analyses.
SHAP or LIME	SHAP bar, beeswarm and waterfall plots at six horizons.
Hazardous AQI alerts	Dashboard category and health-alert logic.
Interactive dashboard	Public Streamlit Community Cloud deployment.
Detailed report	This document and repository README.

The project therefore covers the full functional scope of the supplied brief, including the automation requirements that distinguish an end-to-end system from a static forecasting experiment.

17. Limitations and Future Work

The current system is complete for the internship submission, but several limitations are material. First, the selected air-quality source is model-based rather than an official Karachi station network. Second, historical training uses realized target-hour weather while production uses forecast weather, so training and serving do not have identical covariates. Third, long-horizon accuracy is lower than short-horizon accuracy and rare severe AQI episodes are under-represented. Fourth, the dashboard

publishing bridge uses an hourly CSV commit because it is simple and serverless; a larger production system would normally serve forecast records through an API, database or object store.

The next technical improvements would be to archive historical weather forecasts for train/serve parity, add uncertainty intervals, score performance separately on unhealthy/hazardous episodes, add feature and error drift monitoring, introduce model-promotion rules rather than automatically using every newly trained version, and evaluate additional ground-station data when available. The same architecture could also be parameterized for multiple cities.

18. Conclusion

The Karachi AQI Predictor evolved from a forecasting exercise into an operational ML system. The strongest part of the project is the connection between analysis and engineering: EDA led to longer temporal features and weather context; baseline failures at long horizons motivated more expressive models; SHAP showed the expected change from persistence to meteorology as the horizon increased; and deployment failures led to concrete reliability features such as gap repair, delayed targets, model version selection and environment isolation.

The final system continuously acquires data, reconstructs features, predicts the next 72 hours, exposes health-oriented AQI information in a public dashboard, accumulates labels as time passes, retrains daily and versions the resulting models. In that sense, the project meets the original brief at the system level rather than only at the model level.

References

1. 10 Pearls. Pearls AQI Predictor project brief, supplied for the internship project, 2026.

2. Open-Meteo. Weather API, Historical Weather API and Air Quality API documentation. https://open-meteo.com/

3. Hopsworks. Feature Store and Model Registry documentation. https://docs.hopsworks.ai/

4. XGBoost documentation. https://xgboost.readthedocs.io/

5. SHAP documentation. https://shap.readthedocs.io/

6. GitHub Docs. GitHub Actions and scheduled workflows. https://docs.github.com/actions

7. Streamlit Docs. Streamlit Community Cloud deployment documentation. https://docs.streamlit.io/deploy/streamlit-community-cloud

Appendix A - Repository Script Map

File	Role
1_collect_data.py	Historical Open-Meteo air-quality and weather download.
2_prepare_data.py	Cleaning, continuity checks and final feature engineering.
3_train_models.py	Baselines, Ridge and Random Forest comparison.
4_train_tensorflow.py	Dense neural-network experiment.
5_train_hourly_xgboost.py	Initial direct 1-72 hour XGBoost training.
6_shap_explanations.py	SHAP explanations at representative horizons.

7_upload_features_to_hopsworks.py	Historical feature-group upload.
8_upload_targets_to_hopsworks.py	Historical target-group upload.
9_create_feature_view.py	Hopsworks Feature View/training data setup.
10_train_from_hopsworks.py	Initial training-from-Feature-Store validation.
11_train_final_models_from_hopsworks.py	Current daily 72-model training pipeline.
12_register_models_hopsworks.py	Build and register versioned ZIP model artifact.
13_live_feature_pipeline.py	Hourly self-healing feature ingestion and upsert.
14_predict_live.py	Latest-model retrieval and live 72-hour inference.
15_dashboard.py	Streamlit dashboard.
16_update_targets.py	Delayed target generation for daily retraining.
eda.py	Exploratory analysis and EDA plots.